

Hyperparameter Tuning

Lecture 8
IIT Kanpur

Content Copyrights

- I own the copyright of the CS776: Deep Learning for Computer Vision course material. It includes lectures, presentations, exams and assignments. It should not be distributed in print or through electronic media without the consent of the instructor. Students can take notes or make their own copies of the content.

Performance metrics

- Measuring accuracy

- $Accuracy = \frac{\text{Correct predictions}}{\text{total number of examples}}$

- Accuracy is not always the best metrics to evaluate the model performance

- Confusion metrics

True Positive	False Negative
False Positive	True Negative

Example

- Classifier to predict if the person is sick or not
 - *positive as in the patient is sick, or*
 - *negative, the patient is healthy*

	Predicted sick (positive)	Predicted healthy (negative)
Sick Patients (positive)	100 True Positives (TP)	30 False Negative (FN)
Healthy Patients (negative)	70 False Positives (FP)	800 True Negatives (TN)

- true positives (TP): These are cases in which the model has correctly predicted yes (they have the disease).
- true negatives (TN): the model correctly predicted no disease.
- false positives (FP): the model falsely predicted yes, but they don't actually have the disease ("Type I error" or "error of the first kind").
- false negatives (FN): the model falsely predicted no, but they actually do have the disease ("Type II error" or "error of the second kind")

Performance metrics

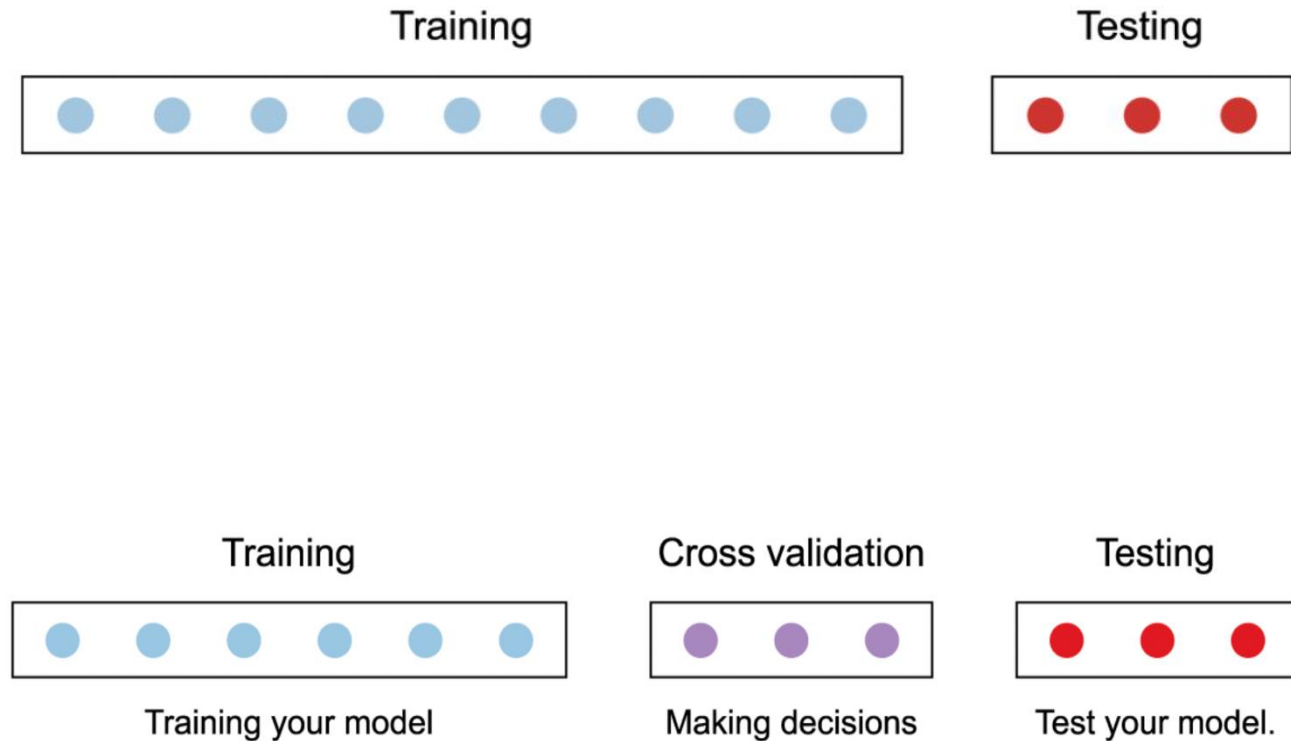
- Recall – sensitivity
 - How many times the model incorrectly predicted negative (false negative)
 - how many actual positives were found
 - $Recall = \frac{true\ positive}{true\ positive + false\ negative}$
- Precision – specificity
 - how many predicted positives were correct
 - $Precision = \frac{true\ positive}{true\ positive + false\ positive}$
- F1-score
 - Harmonic mean of precision and recall
 - $F - score = \frac{2pr}{p+r}$

Baseline Model Designing

- Model architecture
 - CNN/MLP/others
 - Number of layers
 - Neurons in each layer
 - Activation function
 - Optimizer
- Existing models can be used as a starting point

Data Preparation

- Splitting data to train/test/validate
 - Training and Testing data should be always different
 - Testing the model on unseen data provides unbiased performance evaluation
- Validation dataset
 - Used to calculate error and accuracy of the model during training
 - Used to tune the model parameters

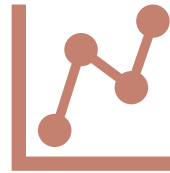


Data splitting



If the dataset is small (around 10000 images/data points)

train/validate/test -> 70/15/15 or 60/20/20



If the dataset is large (more than 1 million samples)

1% data for both testing and validation is sufficient

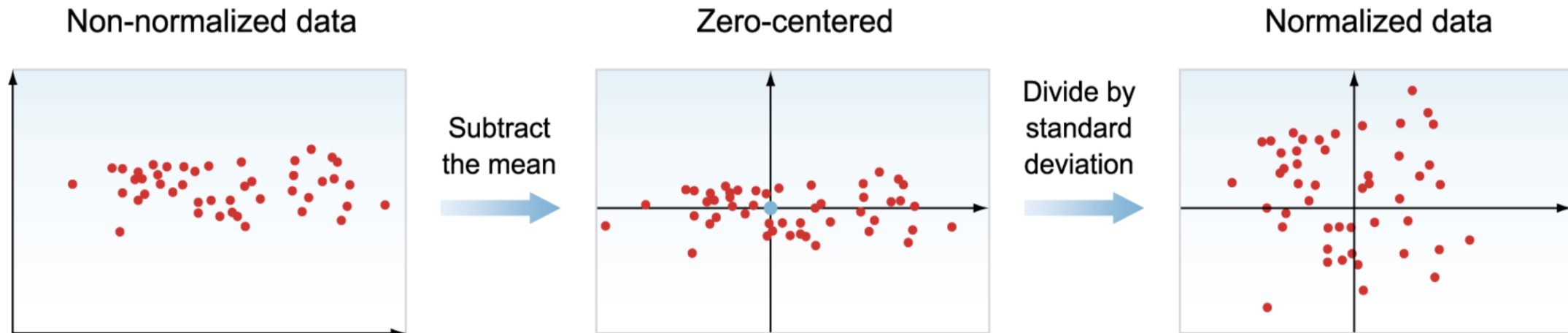


Datasets from the same distribution

image resolutions
h/w for development and deployment

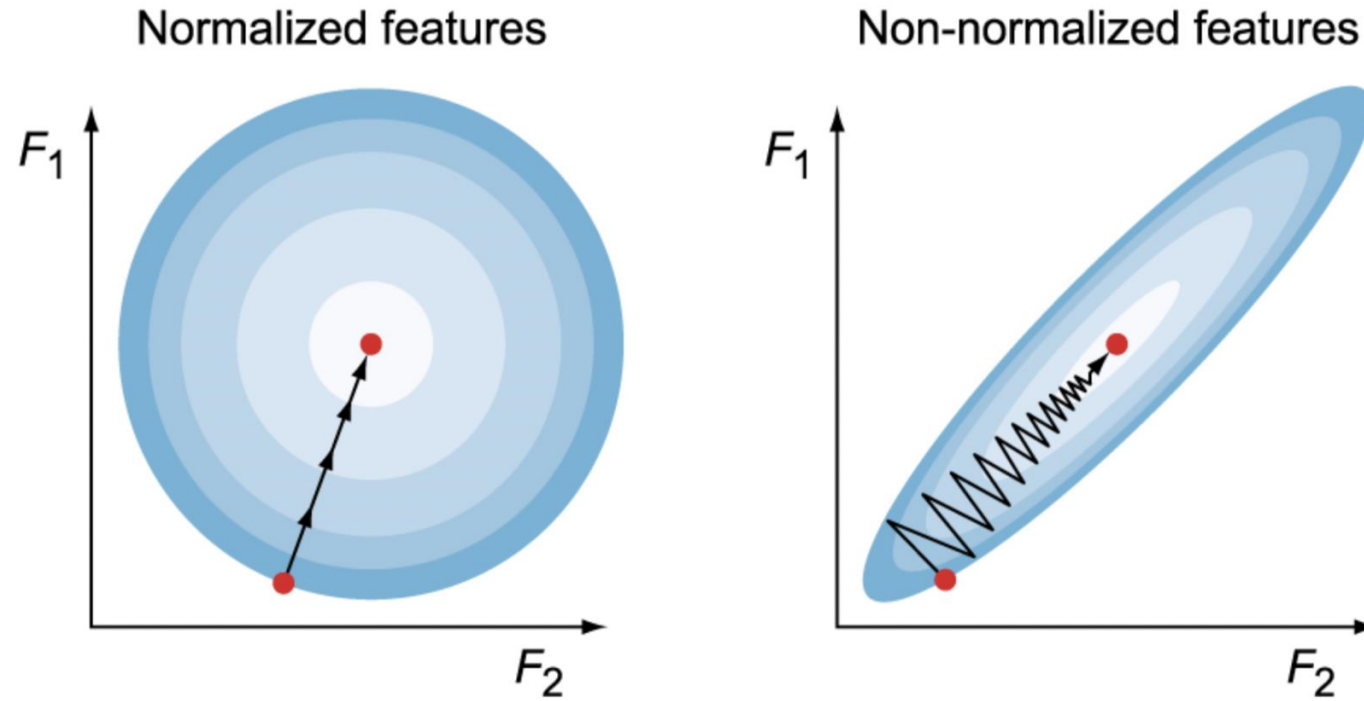
Data preprocessing

- Image gray scaling
 - Reduces computation as compared to color images
 - Application dependent
- Image resizing
 - CNNs require input images of the same size
- Data Normalization
 - Ensures similar data distribution (values in the same range)



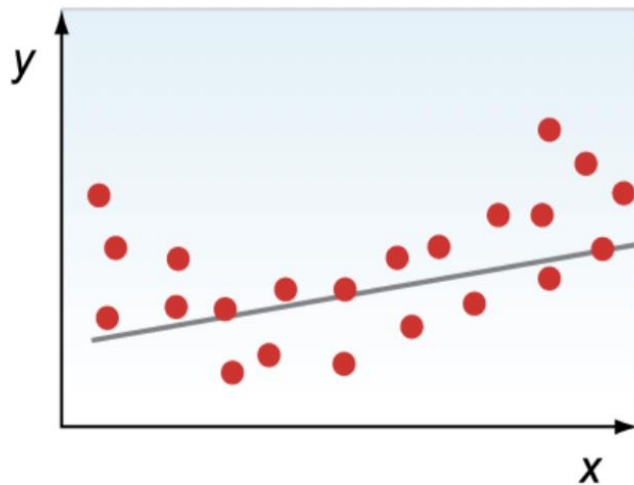
Data normalization

Gradient descent with and without feature scaling



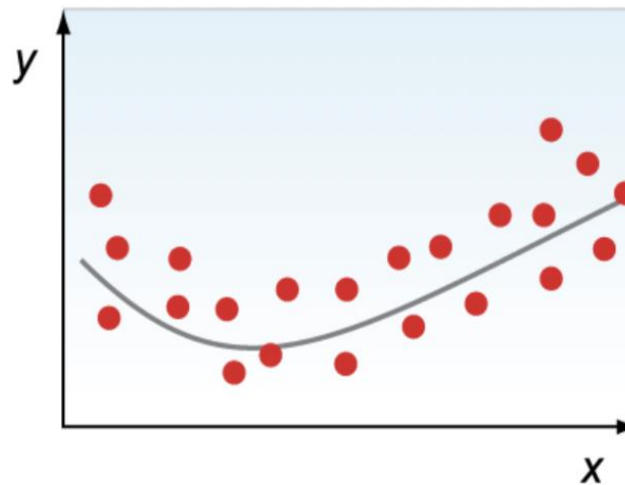
Evaluating model and its performance

Underfitting



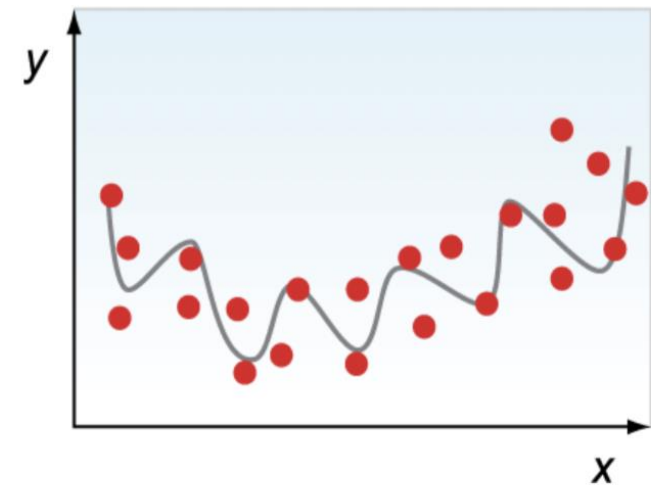
- Fails to fit the training data
- Simple model

Just right!



- Fits training data well
- Predicts well on unseen data

Overfitting

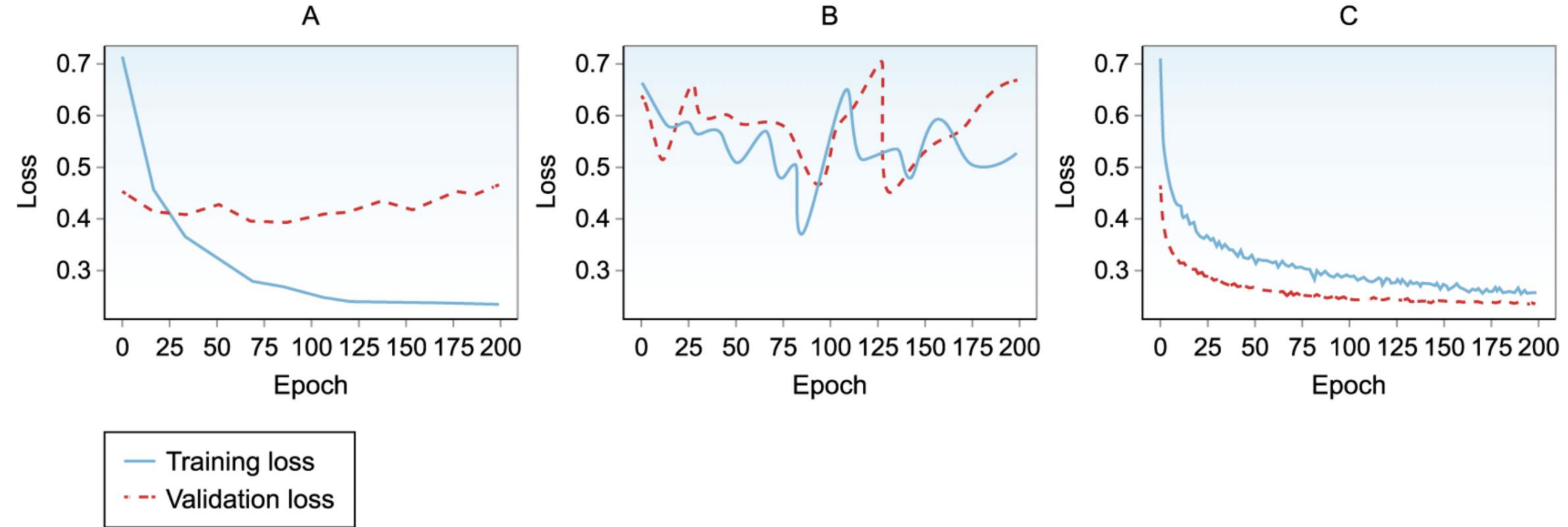


- Fitting the data too much
- Memorizing training data, no feature learning

Diagnosing overfitting and underfitting

- Overfitting – if model performs very well on training set but gives substantial error for validation set
 - Solution – tune hyperparameters, iteratively train/validate/test
- Underfitting – if model performs poorly on the training dataset
 - Solution – add more hidden layers, increase number of neurons
- Use human-level performance to identify a Bayes error rate
 - Bayes error rate – best possible error rate
 - Zero error rate is not practical

Plotting the learning curves



Improving the Network and tuning hyperparameters

- More training data
 - Collecting more training data can be expensive e.g. for medical images
- Gathering more data vs tuning hyperparameters
 - Required accuracy
 - Train vs validation accuracy
 - Overfitting or underfitting

Parameters vs Hyperparameters

- Hyperparameters
 - Manually set and tuned
 - E.g. Learning rate, epochs, batch size, hidden units etc
- Parameters
 - Learning during the model training
 - E.g. Weights, biases etc

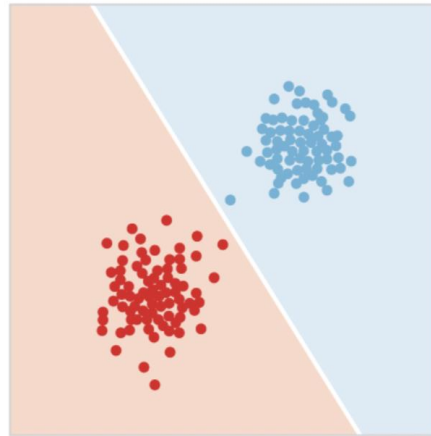
Hyperparameter Categorization

- Network architecture
 - Number of hidden layers (network depth)
 - Number of neurons in each layer (layer width)
 - Activation type
- Learning and optimization
 - Learning rate and decay schedule
 - Mini-batch size
 - Optimization algorithms
 - Number of training iterations or epochs
- Regularization techniques to avoid overfitting
 - L2 regularization
 - Dropout layers
 - Data augmentation

Network Architecture

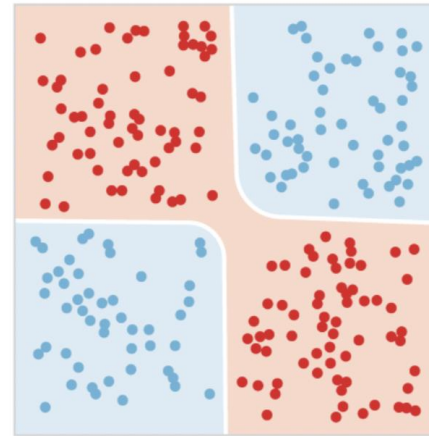
- Number of hidden layers (network depth)
- Number of neurons in each layer (layer width)
- Activation type

Very simple dataset



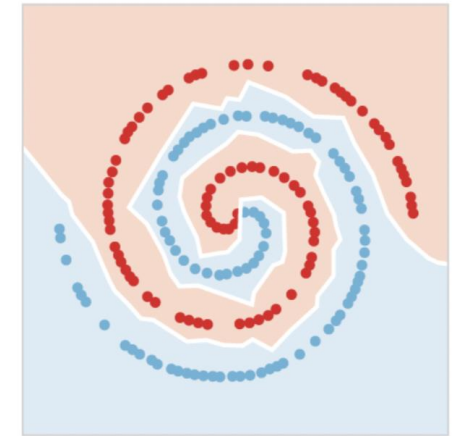
Can be separated by a single perceptron

Medium complexity dataset



Can be separated by adding a few more neurons

Complex dataset



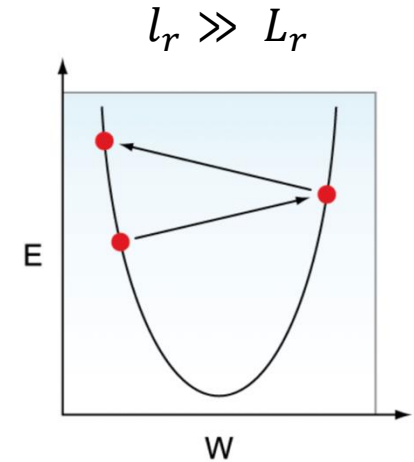
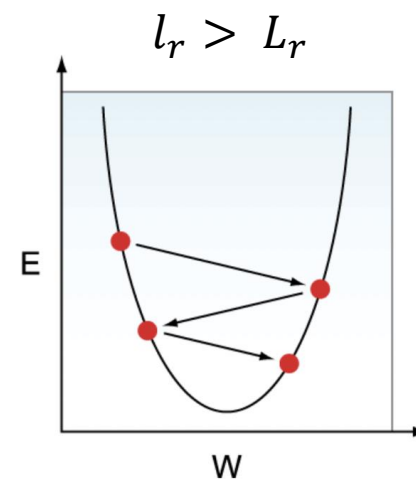
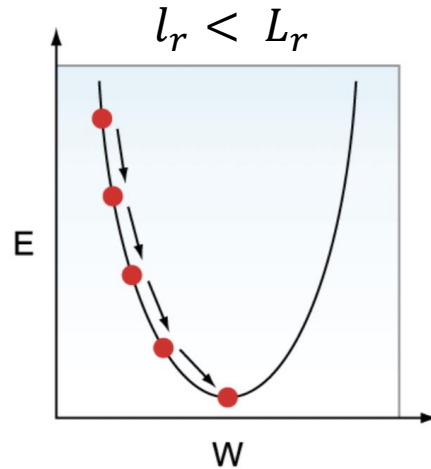
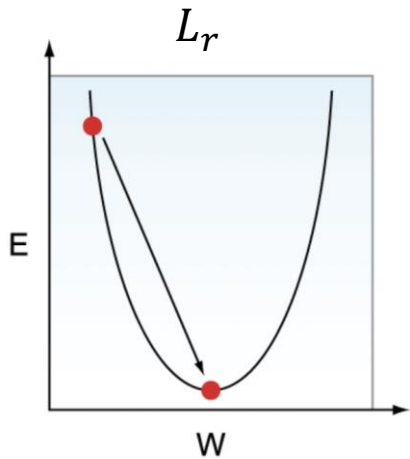
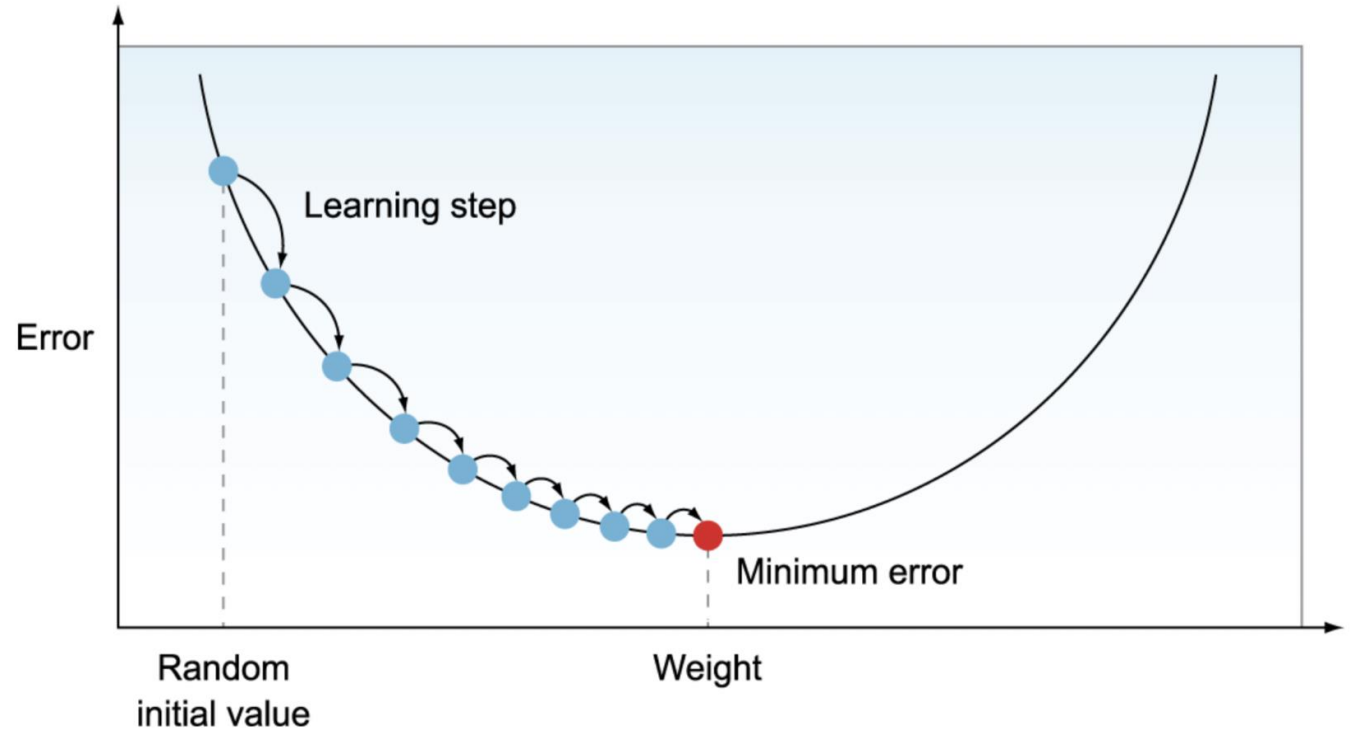
Needs a lot of neurons to separate the data

Learning and Optimization

- Learning rate and decay schedule
- Mini-batch size
- Optimization algorithms
- Number of training iterations or epochs

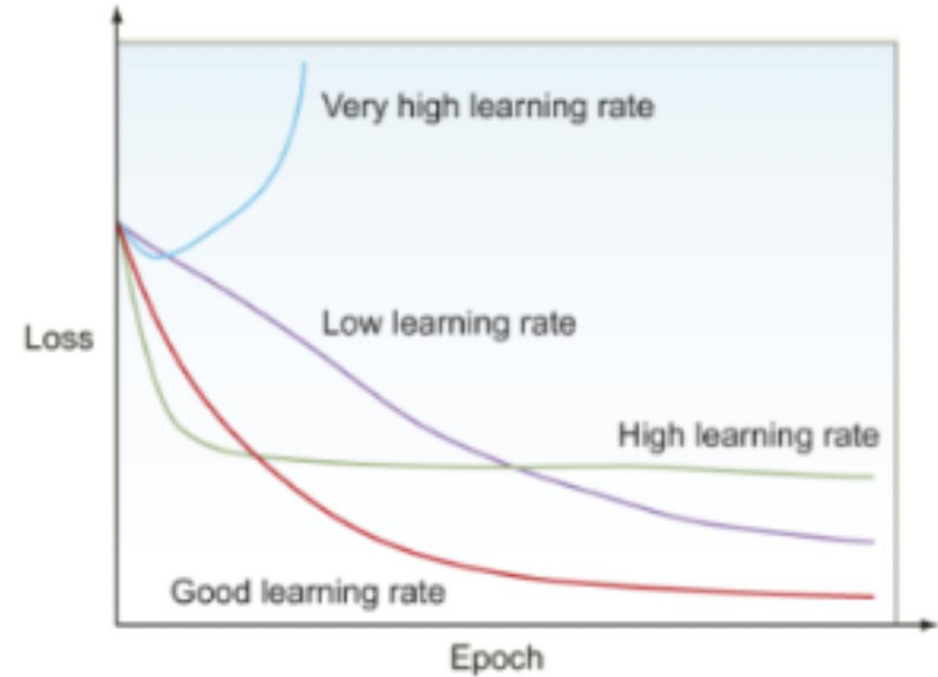
Learning rate

- A step size to descend the error curve
- Decides the speed of the model training
- L_r - ideal learning rate
- l_r - selected learning rate



Learning rate

- Very small l_r - large time to converge
- Larger l_r - does not reach the optimal point
- Very high l_r - divergence
- Good l_r - converges to optimal point with consistent decrease in loss

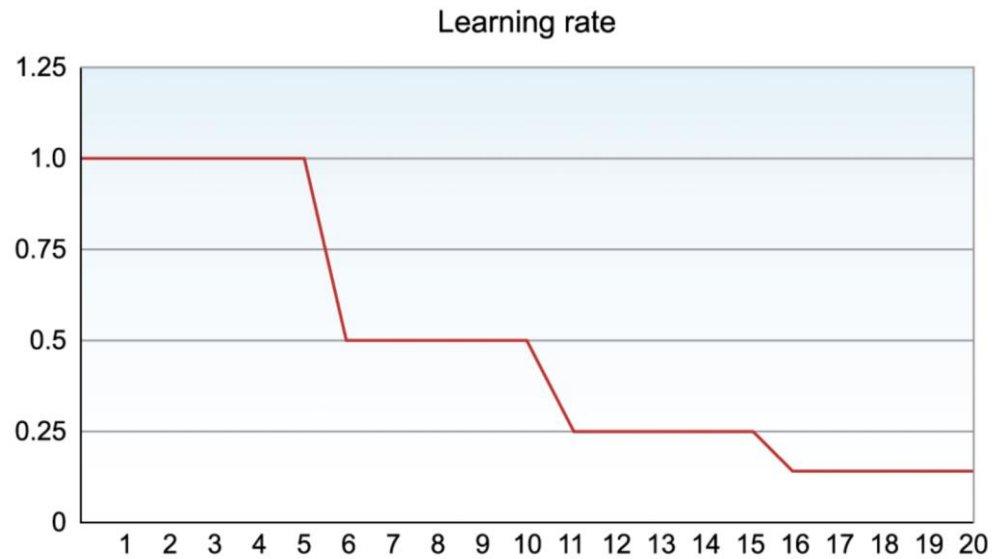


Optimal learning rate

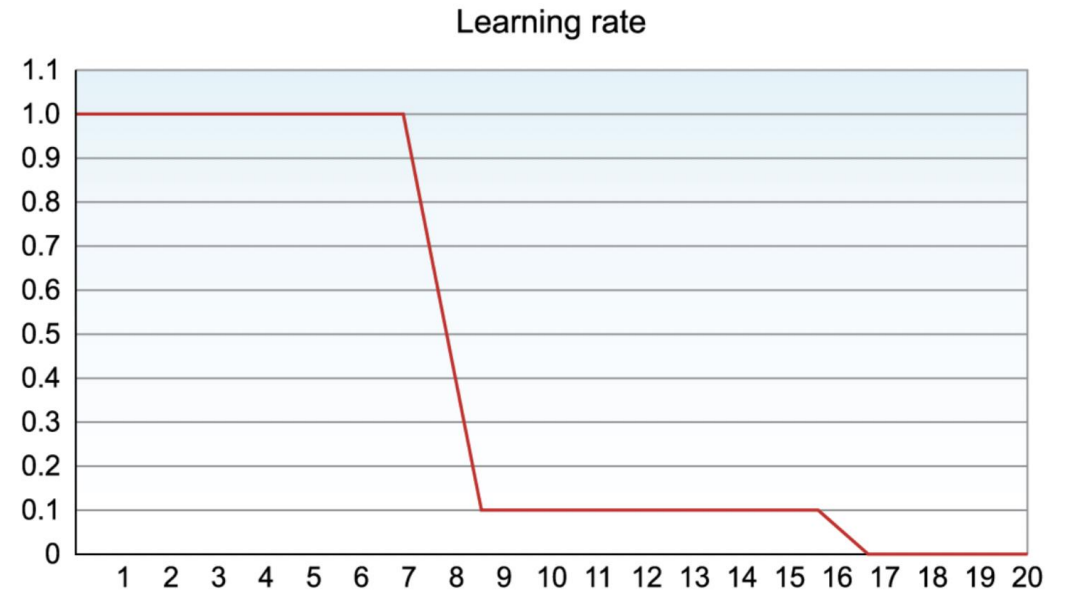
- Depends on model architecture and the dataset
- Check the validation loss
 - If validation loss keeps on decreasing, continue training with the same l_r
 - If training is complete and validation loss keeps on decreasing,
 - Training with more number of epochs or
 - Increase the l_r value
 - If validation loss increases or oscillates, reduce l_r value.

Learning decay

- Reduces learning rate during training



Linear decay – reduce l_r by half after every 5 epochs



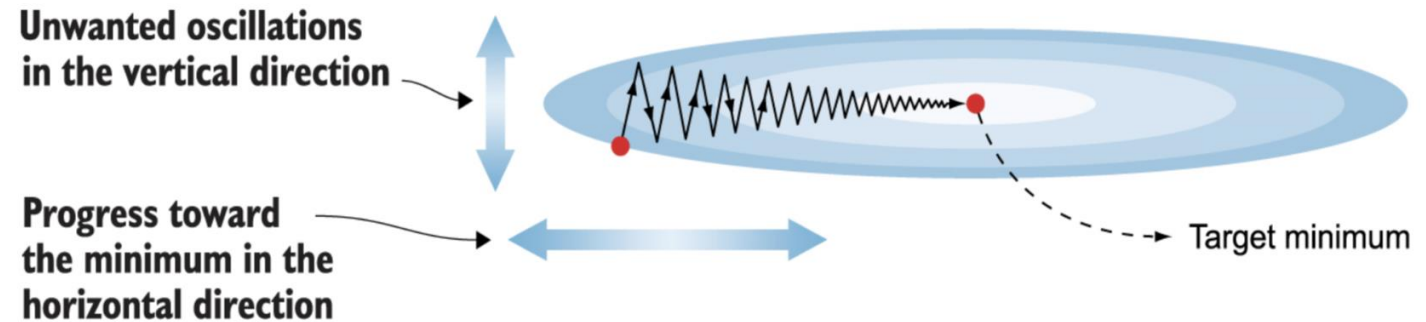
Exponential decay – Multiply l_r by 0.1 for every 8 epochs

Mini-batch size

- BGD – Batch Gradient Descent
 - Updates weights for the entire dataset
 - Advantage – low noise
 - Disadvantage – requires more memory
- SGD – Stochastic Gradient Descent
 - Weights are updated after processing one sample
 - Advantage - performs better than BGD
 - Disadvantage – compromises on the speed gain from the matrix multiplications
- MB-GD – Mini-batch gradient descent
 - Training dataset is divided into smaller batches
 - Advantage – faster training with the speed gain from the matrix multiplications
- Choosing the batch size
 - Dependent on the CPU/GPU memory

Optimization Algorithms

- Gradient descent with momentum
 - Original weight update rule
 - $W_{new} = W_{old} - \alpha \frac{dE}{d_{w_i}}$
 - Weight update after adding the velocity term
 - $W_{new} = W_{old} - \alpha \frac{dE}{d_{w_i}} +$
velocity term
 - *velocity term* ==
weighted average of the
past gradients



Optimization Algorithms

- Adam – Adaptive momentum estimation
 - Considers exponentially decaying average of past gradients
 - Trains the network quickly as compared to other optimizers
 - Controls the momentum
 - `keras.optimizers.Adam(lr=0.001, beta_1=0.9, beta_2=0.999, epsilon=None, decay=0.0)`
 - For the momentum term β_1 , a common choice is 0.9.
 - For the RMSprop term β_2 , a common choice is 0.999.
 - ϵ is set to 10^{-8} .
 - No need to train the Adam parameters. Default values work quite well

Epochs

- Number of epochs required for model training can be decided based on the training and validation error values
- Example

Epoch 1, Training Error: 5.4353, Validation Error: 5.6394

Epoch 2, Training Error: 5.1364, Validation Error: 5.2216

Epoch 3, Training Error: 4.7343, Validation Error: 4.8337

Continue training since both training and validation errors are reducing

Epoch 6, Training Error: 3.7312, Validation Error: 3.8324

Epoch 7, Training Error: 3.5324, Validation Error: 3.7215

Epoch 8, Training Error: 3.7343, Validation Error: 3.8337

Make the model complex since both training and validation errors are fluctuating

Early Stopping

- Stops the training if training error is improving but validation error is increasing (model overfitting)
- Determines the right time to stop the training
- *EarlyStopping*(*monitor* = ' *val_loss*, *min_delta* = 0, *patience* = 20)
 - *monitor* – monitors the validation loss
 - *min_delta* – minimum change that qualifies as an improvement. Default value is 0.
 - *patience* – number of epochs to stop the training after detecting the error is not improving

Regularization Techniques

- L2 regularization

- Simplifies the model by reducing weight values of the hidden layers (close to zero)
- Updates the error function
 - $error\ function_{new} = error\ function_{old} + regularization\ term$
 - $regularization\ term = \frac{\lambda}{2m} \times \sum ||w||^2$
 - λ - regularization parameter,
 - m - number of instances, and
 - w - weight.
 - Updated error function
 - $error\ function_{new} = error\ function_{old} + \frac{\lambda}{2m} \times \sum ||w||^2$
 - $W_new = w_old - \alpha * d(E + \frac{\lambda}{2m} \times \sum ||w||^2) / dw$

- L1 regularization

- $error\ function_{new} = error\ function_{old} + \frac{\lambda}{2m} \times \sum ||w||$
- Makes the w vector sparse – large numbers of weights will have value zero

Regularization Techniques - L2 regularization

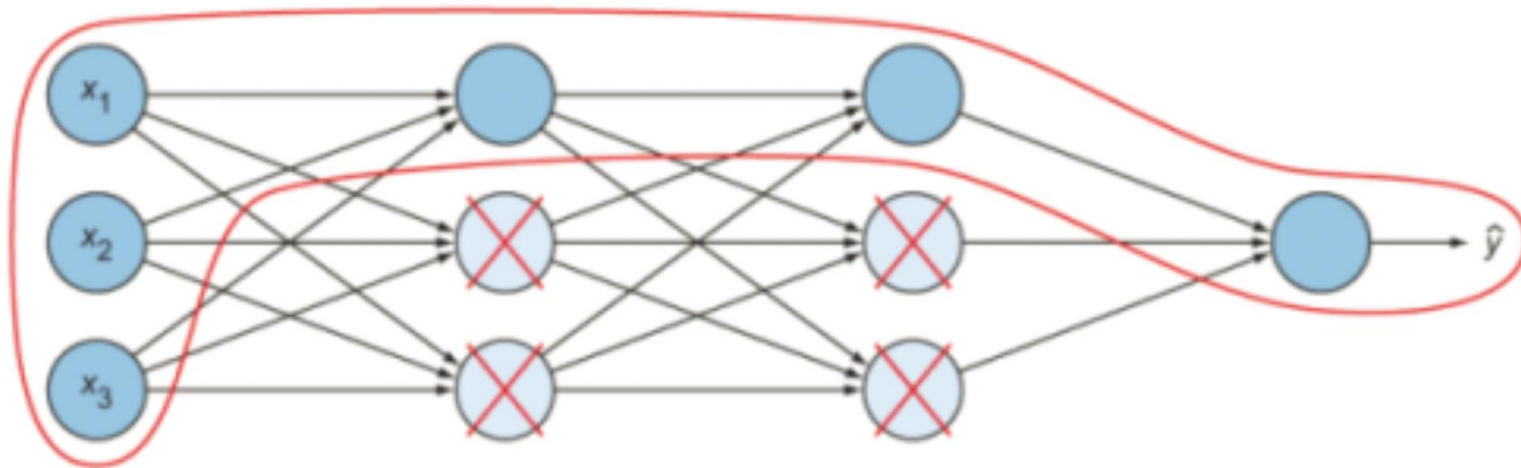
- For the larger value of the regularization term (λ), the new weight value becomes close to zero. Results in reduced model complexity
- Also known as weight decay

Diagram illustrating the L2 regularization formula:

$$W_{new} = W_{old} - \alpha \left(\frac{\partial Error}{\partial W_x} \right)$$

Labels and arrows:

- New weight** points to W_{new}
- Old weight** points to W_{old}
- Learning rate** points to α
- Derivative of error with respect to weight** points to $\frac{\partial Error}{\partial W_x}$

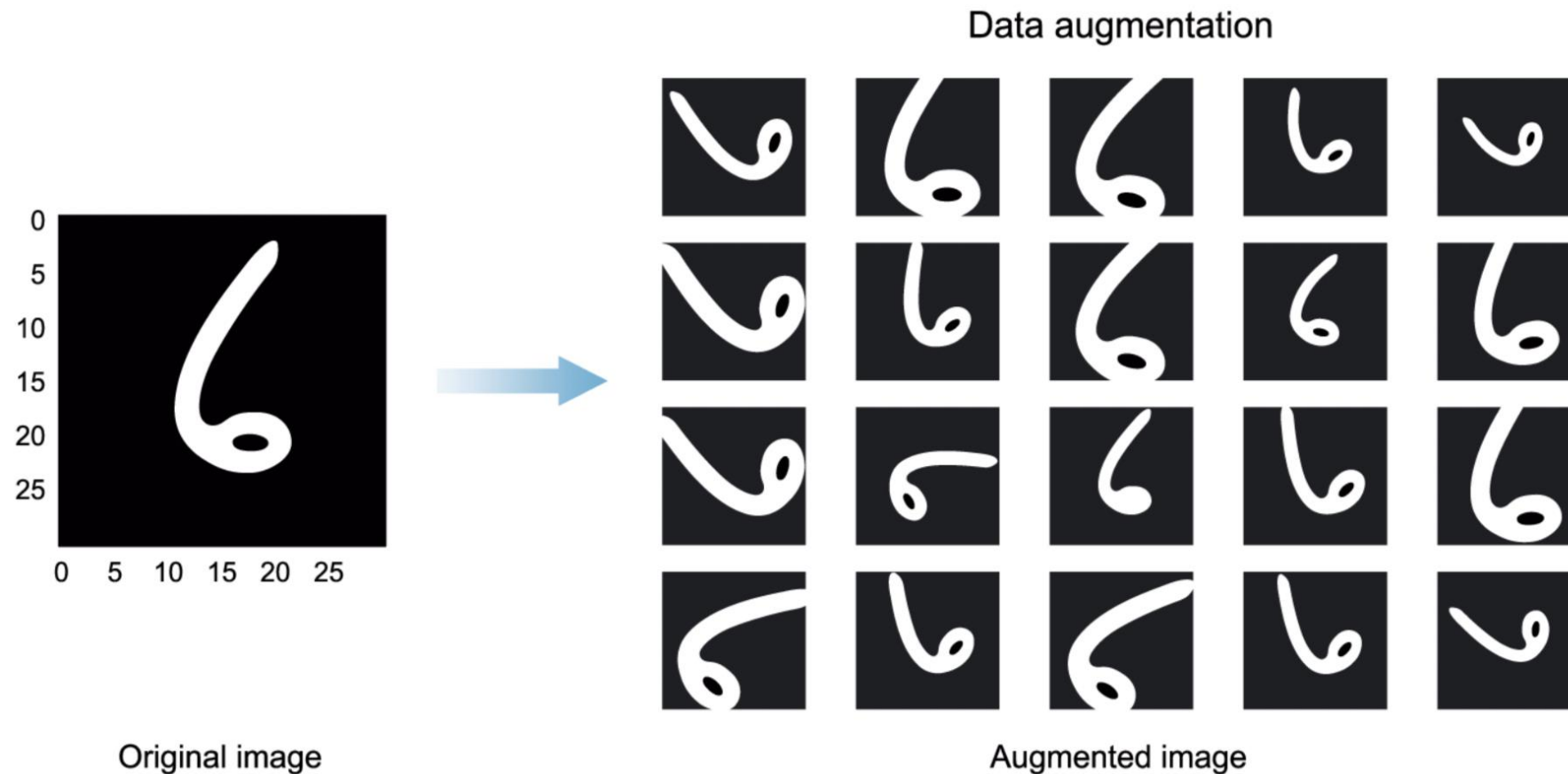


Regularization Techniques - Dropout layers

- Randomly drops out different neurons in each iteration during training.
- Simplifies the network by reducing the number of participating neurons in the training
- Recommended to use both L2 and dropout techniques in model training

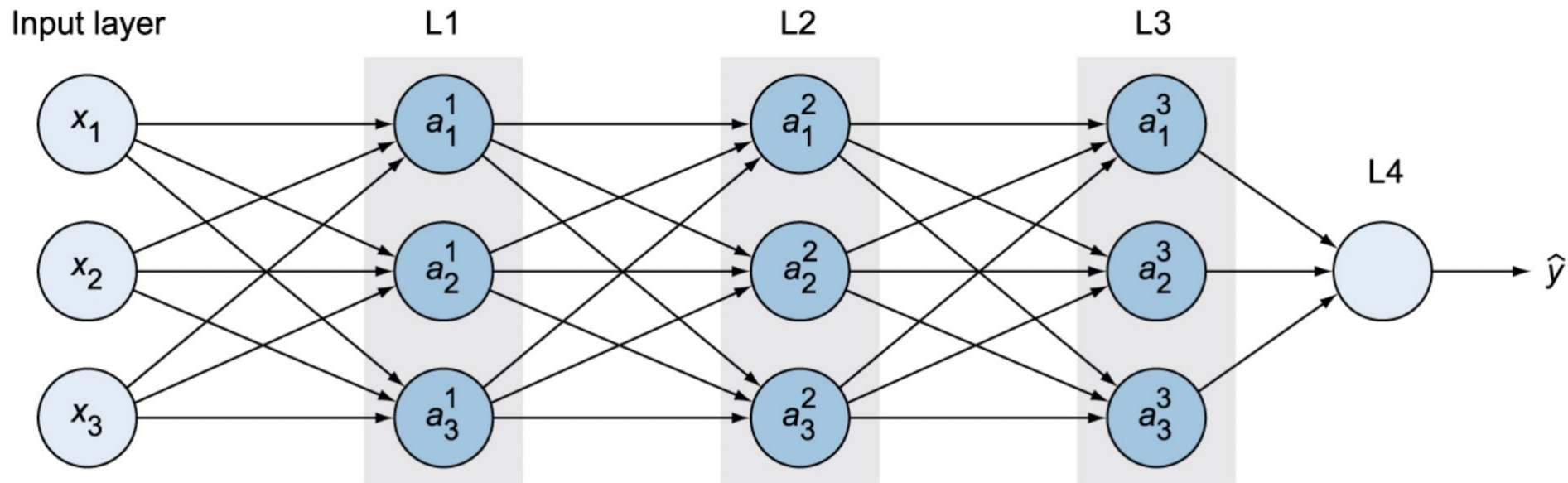
Regularization Techniques- Data Augmentation

Makes the model resilient on new test data



Batch Normalization

- Normalizes data just before the activation function for every hidden layer

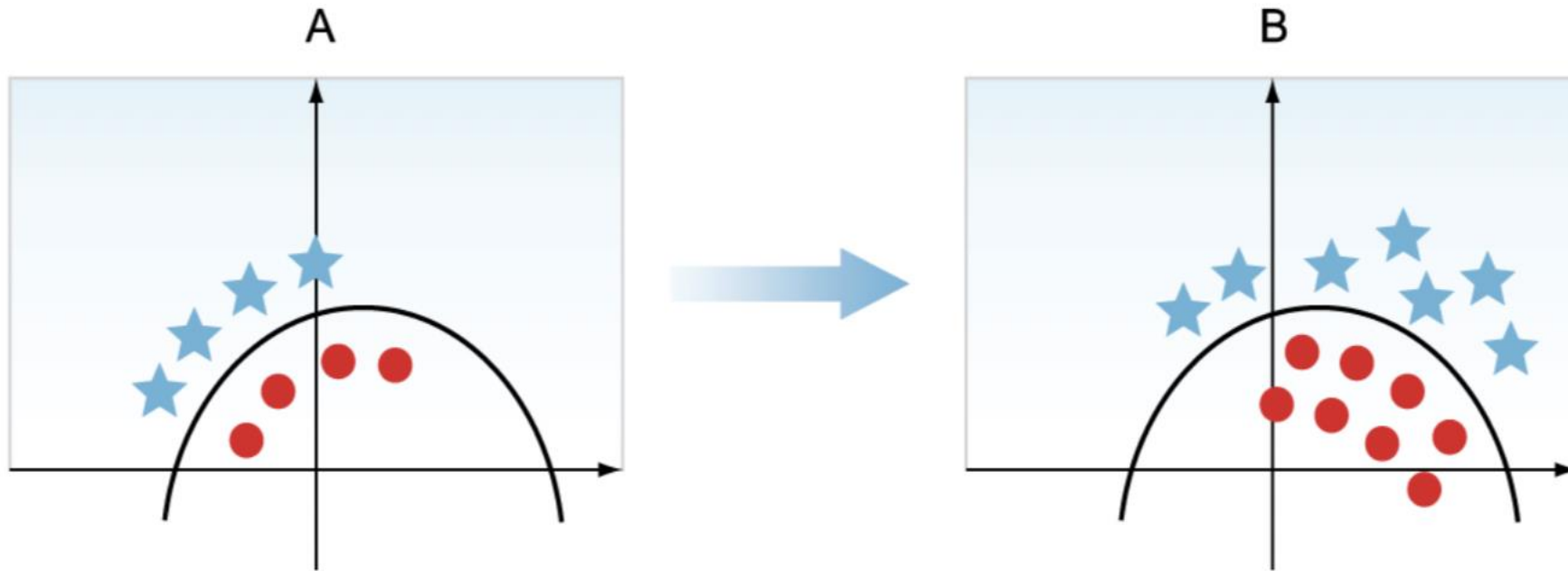


These activations are essentially the input to the following layers, so why not normalize these values?

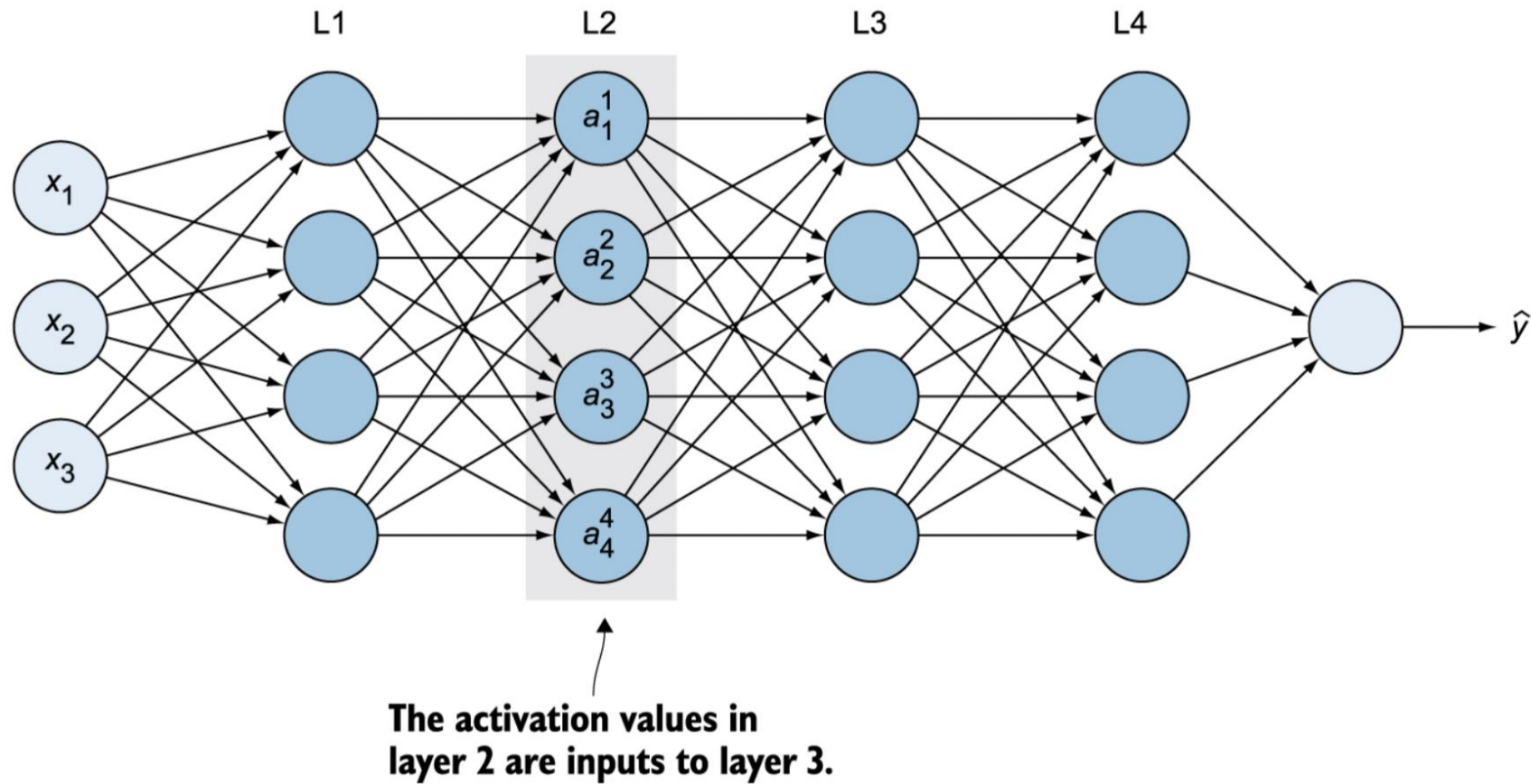
Covariate shift problem

- Change in data distribution

Covariance shift



Covariate shift in neural network



How batch normalization works

- Normalizes data just before the activation function for every hidden layer
- Keeps the mean and variance same
- Batch-norm operations
 - Zero-center the inputs
 - Mini-batch mean

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$$

- Mini-batch variance

$$\sigma_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

- Normalize the inputs

$$\hat{x}_B \leftarrow \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

- Scale and shift the results

$$y_i = \gamma x_i + \beta$$

Keras implementation of batch normalization

```
from keras.models import Sequential  
from keras.layers import Dense, Dropout  
from keras.layers.normalization import BatchNormalization
```

```
model = Sequential()
```

```
model.add(Dense(hidden_units, activation='relu'))
```

```
model.add(BatchNormalization())
```

```
model.add(Dropout(0.5))
```

```
model.add(Dense(units, activation='relu'))
```

```
model.add(BatchNormalization())
```

```
model.add(Dense(2, activation='softmax'))
```

Reference

- Book – Deep Learning for Vision Systems by Mohamad Elgendy, Manning Publication, November 2020
- Kingma, Diederik P., and Jimmy Ba. "Adam: A method for stochastic optimization." *arXiv preprint arXiv:1412.6980* (2014).